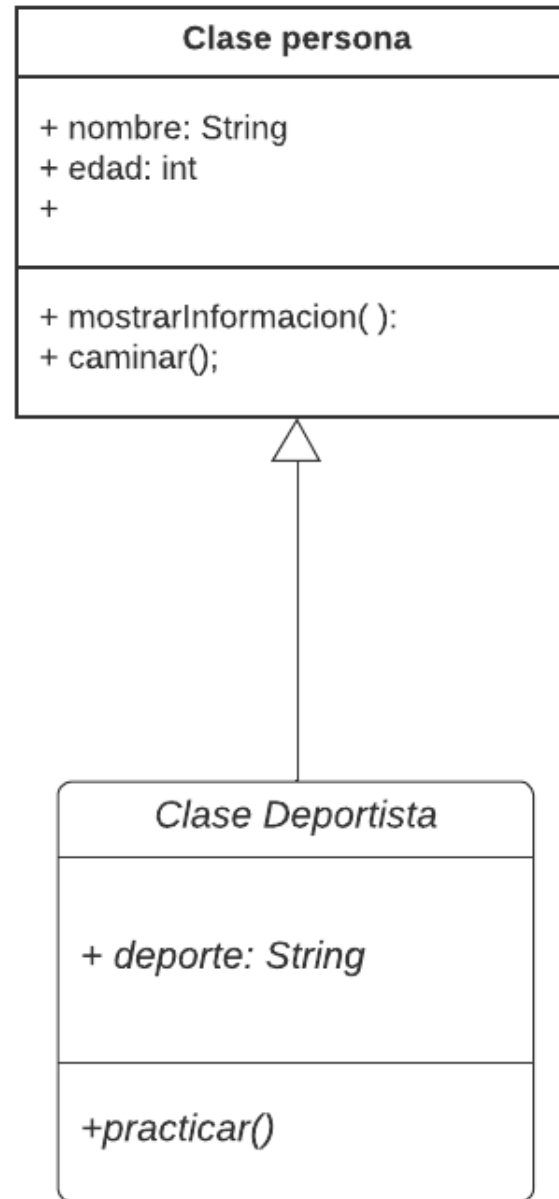


Herencia

Adriana Pérez Espinosa

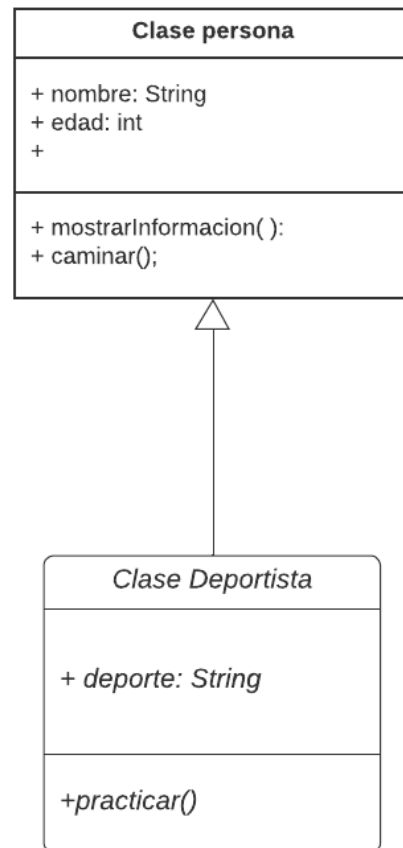


Herencia Simple



Herencia Simple

Diagrama

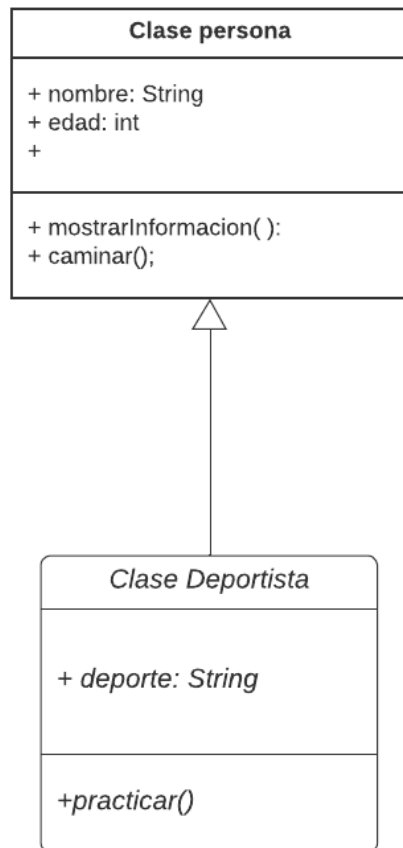


Persona.java

```
1 public class Persona {
2     String nombre;
3     int edad;
4
5
6     public Persona(String nombre, int edad) {
7         this.nombre = nombre;
8         this.edad = edad;
9     }
10
11     public String getNombre() {
12         return nombre;
13     }
14     public void setNombre(String nombre) {
15         this.nombre = nombre;
16     }
17     public int getEdad() {
18         return edad;
19     }
20     public void setEdad(int edad) {
21         this.edad = edad;
22     }
23
24     public void caminar(){
25         System.out.println( nombre+ " esta caminando ");
26     }
27 }
28
```

Herencia Simple

Diagrama

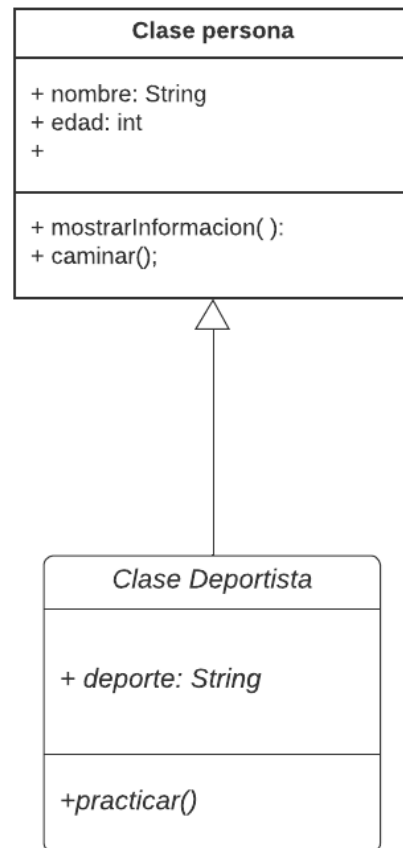


Deportista.java

```
1 public class Deportista extends Persona {
2     String deporte;
3
4
5     public Deportista(String nombre, int edad, String deporte) {
6         super(nombre, edad);
7         this.deporte = deporte;
8     }
9
10
11    public String getDeporte() {
12        return deporte;
13    }
14
15
16    public void setDeporte(String deporte) {
17        this.deporte = deporte;
18    }
19
20    public void practicar() {
21        System.out.println("El deportista " + getNombre() + " practica el deporte "+this.getDeporte());
22    }
23
24 }
```

Herencia

Diagrama



Deportista.java

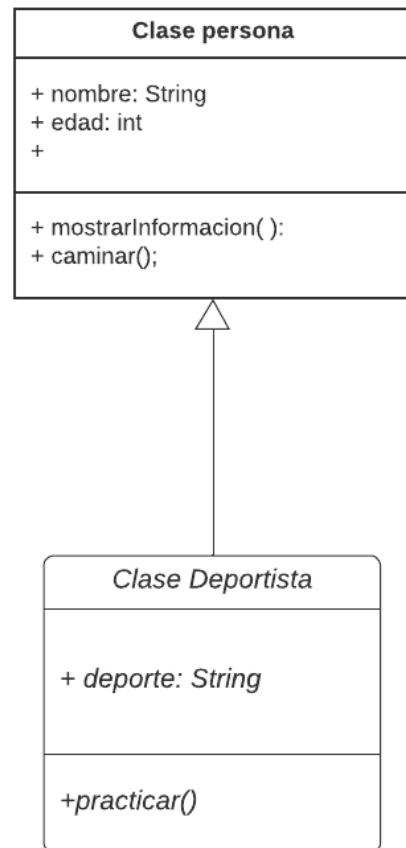
```
1 public class Deportista extends Persona {
2     String deporte;
3
4
5     public Deportista(String nombre, int edad, String deporte) {
6         super(nombre, edad);
7         this.deporte = deporte;
8     }
9
10
11     public String getDeporte() {
12         return deporte;
13     }
14
15
16     public void setDeporte(String deporte) {
17         this.deporte = deporte;
18     }
19
20     public void practicar() {
21         System.out.println("El deportista " + getNombre() + " practica el deporte " + this.getDeporte());
22     }
23
24 }
```

Se utiliza para llamar al constructor de la clase base (padre) desde una subclase (hija).

Herencia Simple

App.java

Diagrama



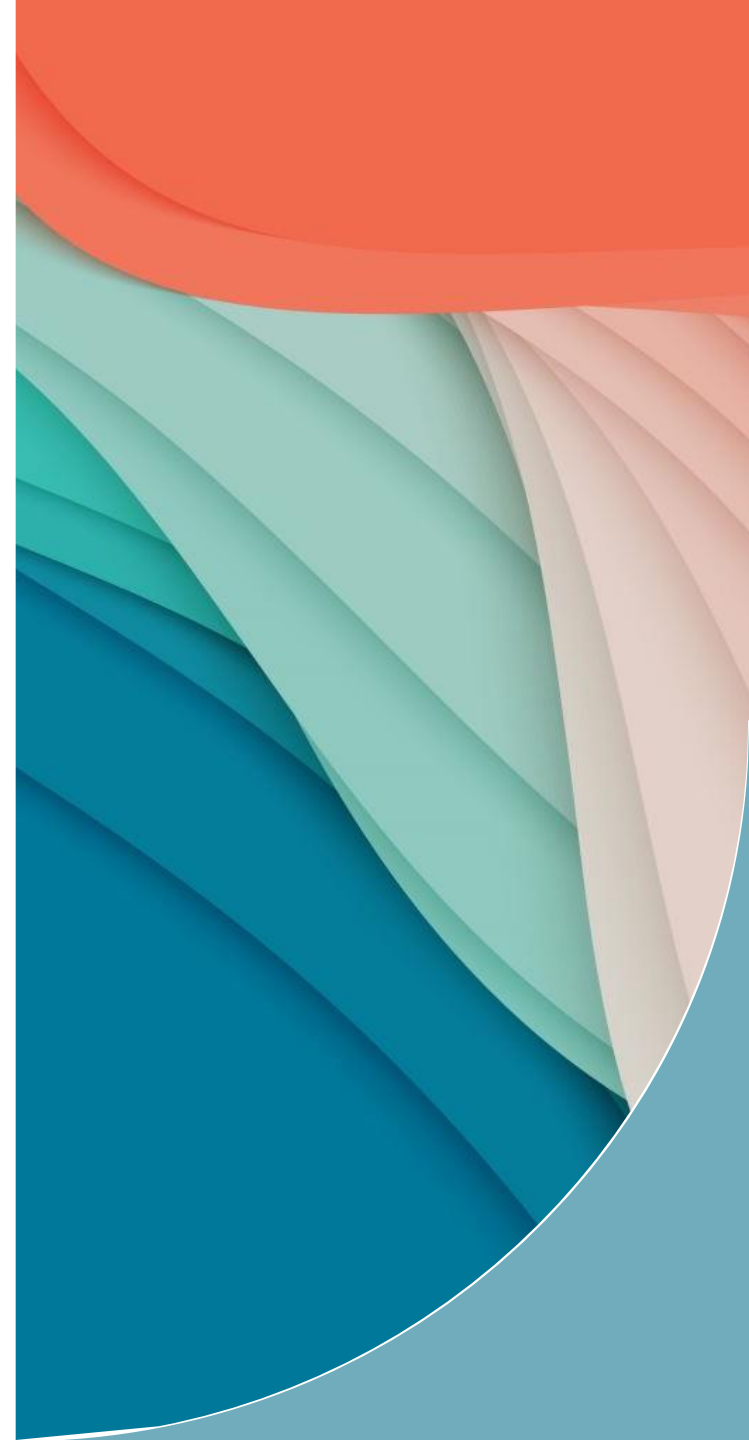
```
1 public class App {
2     public static void main(String [] args){
3         Deportista a= new Deportista("Luis Angel", 54, "natacion");
4         a.caminar();
5         a.practicar();
6     }
7 }
8
```

Salida

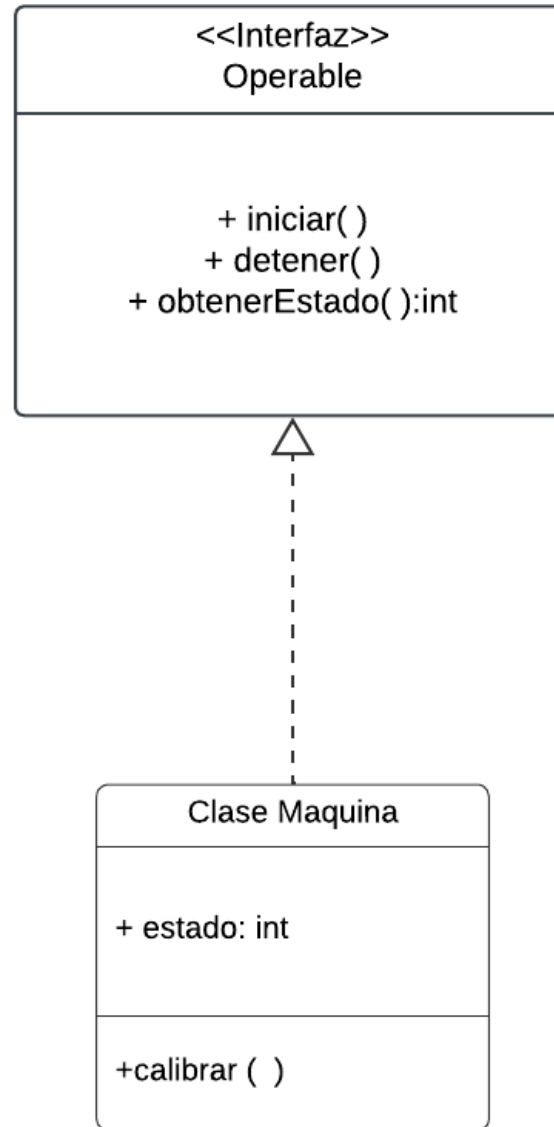
```
Luis Angel esta caminando
El deportista Luis Angel practica el deporte natacion
```

Interfaces

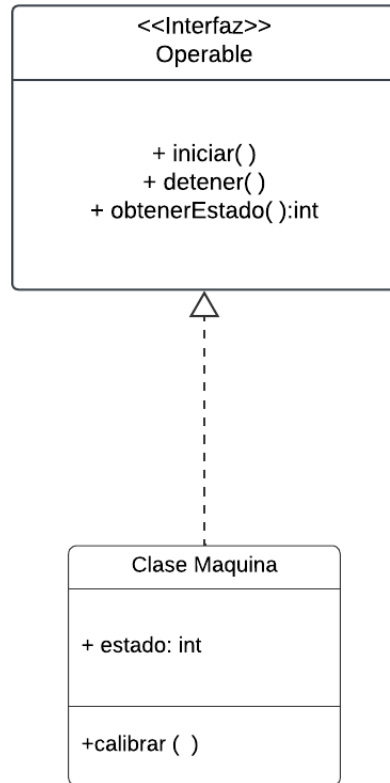
Adriana Pérez Espinosa



Interfaces



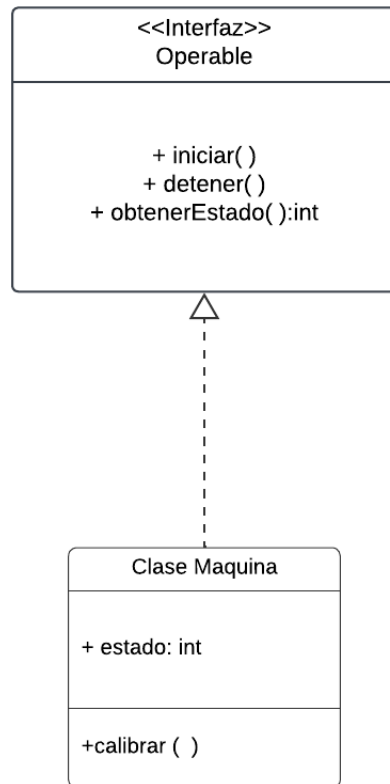
Interfaces



Operable .java

```
1 public interface Operable {
2     void iniciar();        // Método para iniciar la operación
3     void detener();        // Método para detener la operación
4     int obtenerEstado();   // Método para obtener el estado actual
5 }
6
```

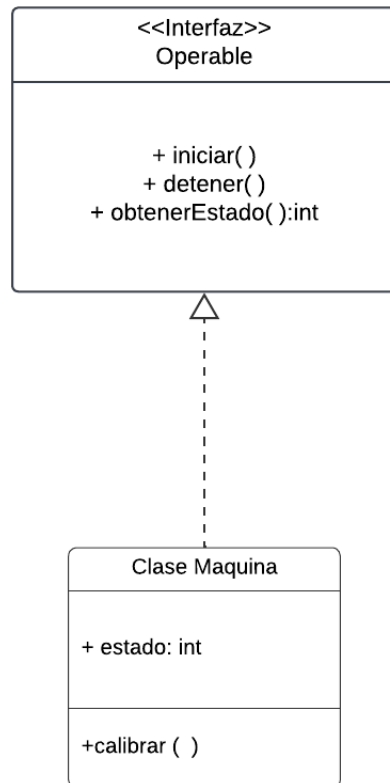
Interfaces



Maquina .java

```
1 public class Maquina implements Operable {
2     private int estado; // 0: Apagada, 1: En funcionamiento
3
4     // Constructor que inicializa el estado de la máquina
5     public Maquina() {
6         this.estado = 0; // La máquina comienza apagada
7         System.out.println("La máquina ha sido creada en estado apagado.");
8     }
9
10    @Override
11    public void iniciar() {
12        estado = 1;
13        System.out.println("La máquina ha iniciado.");
14    }
15
16    @Override
17    public void detener() {
18        estado = 0;
19        System.out.println("La máquina se ha detenido.");
20    }
21
22    @Override
23    public int obtenerEstado() {
24        return estado;
25    }
26
27    // Método adicional específico de la clase Maquina
28    public void calibrar() {
29        System.out.println("La máquina está siendo calibrada.");
30    }
31 }
```

Interfaces

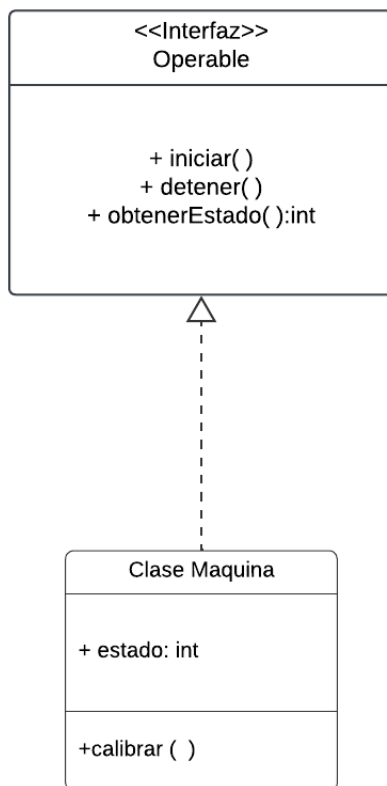


Maquina .java

```
1 public class Maquina implements Operable {
2     private int estado; // 0: Apagada, 1: En funcionamiento
3
4     // Constructor que inicializa el estado de la máquina
5     public Maquina() {
6         this.estado = 0; // La máquina comienza apagada
7         System.out.println("La máquina ha sido creada en estado apagado.");
8     }
9
10    @Override
11    public void iniciar() {
12        estado = 1;
13        System.out.println("La máquina ha iniciado.");
14    }
15
16    @Override
17    public void detener() {
18        estado = 0;
19        System.out.println("La máquina se ha detenido.");
20    }
21
22    @Override
23    public int obtenerEstado() {
24        return estado;
25    }
26
27    // Método adicional específico de la clase Maquina
28    public void calibrar() {
29        System.out.println("La máquina está siendo calibrada.");
30    }
31 }
```

Implementación de los métodos abstractos

Interfaces



App .java

```
1 public class App {
2     public static void main(String[] args) {
3         Maquina maquina = new Maquina(); // Crear una instancia de Maquina
4
5         // Usar los métodos de la interfaz
6         maquina.iniciar();
7         System.out.println("El estado de la máquina: " + maquina.obtenerEstado());
8         maquina.detener();
9
10        // Usar el método adicional específico de Maquina
11        maquina.calibrar();
12    }
13 }
```

Salida

```
La máquina ha sido creada en estado apagado.
La máquina ha iniciado.
El estado de la máquina: 1
La máquina se ha detenido.
La máquina está siendo calibrada.
```